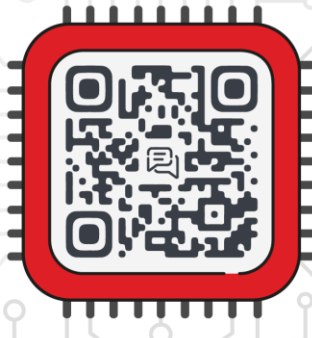




3

8

1200 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Testing

د. سعيد أبو طراب



RB Informatics; 22/04/2026

محتوى مجاني غير مخصص للبيع التجاري

هندسة برمجيات ٢

في دورة حياة تطوير البرمجيات (SDLC)، هناك نشاط مهم جداً يكون موجوداً تقريباً في كل مراحل التطوير، خصوصاً بين مرحلتين: **البناء** (Construction) و **النشر** (Deployment) هذا النشاط هو: **الاختبار** (Testing)

الاختبار (Testing)

في بعض نماذج تطوير البرمجيات، يتم إعطاء الاختبار أهمية كبيرة لدرجة تخصيص مرحلة (Milestone) كاملة له. لكن عملياً، إذا نظرنا الى أرض الواقع داخل الشركات، نلاحظ مشكلة متكررة وهي ان أكثر مرحلتين يتم التقصير فيهما هما:

■ التحليل (Analysis)

■ الاختبار (Testing)

وكان الفريق يقوم "بتجاهل البداية والنهاية"، ويركز فقط على تنفيذ الكود، مع توقع أن النظام سيعمل بشكل صحيح.

في الحقيقة، إن عملية اكتشاف الأخطاء مبكراً عملية مهمة جداً و أرخص في التكلفة لأن:

1. التراجع عنها يكون أسهل.
2. الأخطاء تتكاثر في المراحل المتقدمة.
3. عملية التعديل كلما تقدم المشروع تصبح مكلفة أكثر.
4. اكتشاف الخطأ بعد استلام الزبون للمنتج يكون مكلفاً جداً.



■ ما هو Testing ؟

الاختبار هو عملية تشغيل البرنامج بهدف محدد وهو العثور على الأخطاء قبل تسليمه للمستخدم النهائي. وهنا تشغيل البرنامج لا يعني فتح التطبيق وعمل Run فقط، هذا اسمه تشغيل عادي، اما الاختبار فيبدأ عندما نتفاعل مع النظام وندخل Input ونحصل على Output اي عند وجود Action → Reaction، عندها يُسمى التشغيل بتشغيل التنفيذ قيد الاختبار

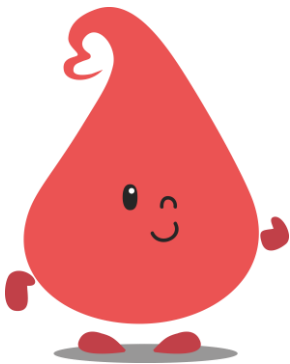
implementation under test (IUT) باستخدام مدخلات الاختبار.

ملاحظة: مراجعة الكود (Code Review) لا تعتبر Testing لأنها لا تعتمد على **التجريب** الفعلي.

■ الهدف من Testing :

هناك أشخاص يعتقدون أن هدف الاختبار هو إثبات صحة البرنامج، لكن هذا الاعتقاد خاطئ لأنه حتى نثبت أن البرنامج صحيح 100% يجب:

تجريب كل القيم الممكنة للإدخال وكل الحالات وكل السيناريوهات، وهذا يؤدي الى عدد لا نهائي من الاختبارات وهذا مستحيل عملياً، بالتالي نستنتج أن Testing لا يهدف لإثبات الصحة، بل لاكتشاف الأخطاء.



فائدة:

إذا قمت بتشغيل Test ولم يظهر خطأ:
فهذا لا يعني أن النظام صحيح
لكنه يعني فقط
"لم أكتشف خطأ في هذه الحالة"
أي يمكن للاختبار أن يُثبت وجود أخطاء ولكنه لا يُثبت غيابها

منظور الشركات:

من وجهة نظر عملية: أي input لا يؤدي إلى اكتشاف خطأ = كلفة مهدورة لأنه استهلك وقت وجهد بدون فائدة مباشرة
>> لذلك المختبر الجيد لا يهدف الى إثبات أن النظام يعمل، بل يهدف الى كسره <<

■ مراحل عملية الاختبار Testing:

2- تنفيذ الاختبار
(Execution)

تشغيل النظام ومقارنة النتائج

1- اختيار المدخلات
(Input Generation)

أصعب جزء ويحتاج تحليل عميق

يتم اختبار التطبيقات عبر مجموعة من حالات الاختبار test cases:
وتتألف كل حالة اختبار من:

test case → (Input + expected output)

Expected Output (الخرج المتوقع):
النتيجة التي نعرف مسبقاً أنها صحيحة
(بناءً على المتطلبات أو الفهم للنظام).

Input (المدخلات):
القيم أو البيانات التي ندخلها للنظام.

ويكون لدينا:

Actual Output (الخرج الفعلي):
النتيجة التي يعطينا إيها النظام فعلياً
بعد التشغيل.

بعد الحصول على النتيجة نقوم بالمقارنة بين:

Actual Output

Expected Output

النتيجة:

- إذا كانوا متطابقين ← الاختبار ناجح
- إذا كانوا مختلفين ← تم اكتشاف خطأ

مثال:

ليكن لدينا تابع بسيط:

$$<< \quad z = x + y \quad >>$$

1. حالة اختبار:

■ Input

$x = 2$ ✓

$y = 3$ ✓

■ Expected Output

$z = 5$ ✓

2. التنفيذ:

■ إذا البرنامج أعطى:

صحيح ← $z = 5$

خطأ ← $z = 6$

وبالتالي في الحالة الطبيعية (تشغيل البرنامج للاستخدام) نحن نفترض أن الـ Output صحيح ، اما في حالة الاختبار لا نفترض أن النتيجة صحيحة بل نقوم بالتحقق منها.

أنواع الأخطاء في البرمجيات (Error / Fault / Failure)

عندما نقول أن "هناك خطأ"، عملياً نكون نتكلم عن 3 مستويات مختلفة:

1. Error (الخطأ البشري):

هو الخطأ الذي يقوم به المبرمج أثناء كتابة الكود، خطأ في التفكير أو في التطبيق

2. Fault (الخلل داخل النظام):

هو الأثر الذي يتركه الـ Error داخل النظام أي أن الكود أصبح في داخله مشكلة لكن لم تظهر بعد للمستخدم وهو "خطأ مخفي"

الفكرة: هناك خطأ في الكود (Error) هذا الخطأ سبب خلل داخلي (Fault) لكن لم يكتشفه أحد.

3. Failure (الفشل الظاهر):

هو ظهور الخطأ أثناء التشغيل وملاحظته.

مثال للتوضيح:

لدينا الشرط $if(x \leq 3)$ والمطور اثناء كتابة الـ code نسي الـ "=" فكتب الشرط كالتالي: $if(x < 3)$ يعتبر هذا error وهذا الـ error سبب fault لم يظهر لأنه لم تصل قيمة الـ $x=3$ ولكن عندما تصبح قيمة $x=3$ ويظهر الخطأ هنا يصبح failure.

إنّ الهدف من Test cases هو اكتشاف أكبر قدر ممكن من الـ faults قبل ان تصبح failures ووصولها للمستخدم

لأن:

- اكتشاف الخطأ بعد التسليم = مكلف جداً
- اكتشافه مبكراً = رخيص

مثال للتكلفة:

- في مرحلة التحليل ← \$ 1
- في مرحلة النشر ← \$ 10-15
- بعد التسليم ← \$ 100



والمشكلة الأكبر أنه ممكن أن يكون هناك عدة أخطاء فوق بعضها، نطلق عليها مصطلح (Fault Masking)

▪ Fault Masking (إخفاء الأخطاء):

هو خطأ يغطي على خطأ ثانٍ وبالتالي النظام يعطي سلوكاً "يبدو طبيعياً" رغم وجود مشاكل داخلية

▪ تحديات ومشاكل الاختبار Testing:

1. Non-Deterministic Systems (عدم الحتمية في التنفيذ)

أي أنه إذا أعدنا تشغيل الاختبار أكثر من مرة لنفس الإدخال يمكن أن نحصل على نتائج مختلفة.
مثال: أنظمة التشغيل (السلوك يعتمد على عوامل لا نلاحظها)

2. عدم الحتمية في المتطلبات:

يحدث هذا إذا كان هناك مدخلات يمكن أن يكون لها أكثر من خرج مقبول
مثال: تابع يقوم بإعادة رقم عشوائي.

3. Fault Masking

وهي حالة الخطأ الذي يخفي خطأ آخر، مما يجعل اكتشاف الأخطاء الجذرية (Root Cause) أكثر صعوبة.

4. Oracle Problem (مشكلة العراف)

وهي مشكلة عدم معرفة Expected Output الصحيح الذي يجب مقارنة النتائج معه

عندما نجري اختبار، نحن دائماً نقارن بين Expected Output & Actual Output ، **لكن السؤال الأهم:**

▪ ما الذي يثبت أن الـ Expected Output نفسه صحيح؟

هنا يظهر مفهوم مهم نسيميه: **Test Oracle** وهو المرجع الذي نستخدمه للحكم على صحة النتيجة
يمكن يكون:

Requirements أو Design أو حتى نظام آخر موثوق و**Oracle Problem** تظهر عند عدم وجود Oracle واضح

مثال:

في الأنظمة المعقدة أو عند وجود خوارزميات كبيرة لا يوجد طريقة سهلة يمكننا معرفة فيما إذا كان الناتج صحيح أم لا، بالتالي وجود Oracle واضح شرط أساسي لنجاح عملية الاختبار.



من يعرف باب الأمل
لا يعرف كلمة مستحيل

V&V -> validation and verification

1. Verification: Are we building the product right?

Verification: هل نحن نقوم ببناء المنتج بشكل صحيح؟ أي التحقق من أن المنتج يقوم بوظائفه بشكل صحيح وكونه يتمتع بمواصفات الجودة سواء على مستوى الـ functional requirements أو على مستوى الـ non-functional requirements.

2. Validation: Are we building the right product?

Validation: هل نحن نقوم ببناء المنتج المطلوب؟ أي التأكد من أن المنتج يحقق المتطلبات ويلبي احتياج الزبون.

مثال:

تخيل أنك طوّرت منتج، والمنتج هو: كرسي

1. السؤال الأول: هل هذا المنتج هو فعلاً "كرسي"؟

بمعنى:

- فيه أرجل؟
- فيه مكان للجلوس؟
- مطابق للتصميم؟



هذا اسمه: Verification

2. السؤال الثاني: هل هذا الكرسي مناسب لاحتياج المستخدم؟

بمعنى:

- مريح؟
- مناسب لطبيعة الاستخدام؟
- يحقق الهدف المطلوب؟



هذا اسمه: Validation

❏ في V&V أين يأتي دور Testing ؟

Testing موجود في الاثنين:

- Verification
- Validation

نختبر أن الكود مطابق للمواصفات وكل Feature تعمل كما هو مطلوب

Verification

نختبر ان النظام فعلاً يخدم المستخدم وهو الحل مناسب

Validation

❏ من هو الـ Tester ؟ وهل يمكن أن يكون هو نفسه الـ Developer ؟

ببساطة الـ Tester هو شخص مسؤول عن اكتشاف الأخطاء في النظام قبل وصولها للمستخدم.

❏ ماذا يجب أن يعرف الـ Tester ؟

الـ Tester الجيد لا يعمل بشكل عشوائي، بل يكون لديه فهم شامل لـ:

- متطلبات العمل (Business Requirements)
- متطلبات النظام (System Requirements)
- الوظائف (Functional)
- الخصائص غير الوظيفية (Non-Functional)
- بنية النظام (Architecture)
- التقنيات المستخدمة (Frameworks)

الـ Tester يرى "الصورة الكاملة" للنظام بينما المطور يركز على جزء محدد من النظام

❏ هل يمكن أن يكون الـ Developer هو نفسه الـ Tester ؟

لا، من الأفضل أن يكون الـ Tester شخص **مستقل** عن فريق التطوير من أجل أن يكون تفكيره مختلف و يرى النظام من زاوية أخرى ويوسع "فضاء الاختبار".

المطور: عندما يختبر يجرب الحالات التي هو يتوقعها ويستمر على سياق منطقته، لكن
المختبر: يحاول كسر هذا المنطق

Tester	Developer	الحالة
 سعيد	 حزين	وجد خطأ
 حزين	 سعيد	لم يجد خطأ

هل يقوم الـ Developer بالاختبار؟

نعم، يقوم بنوع معين نسميه **Unit Testing** (اختبار الوحدات) لكنه اختبار محدود ويركز على أجزاء صغيرة.

**المعرفة سلاح يستعين به الإنسان على فهم
الحياة ومواجهتها وخوض تجاربها...**